

DASS SPRING 2026 · TEAM 8

3D Printing Cost Estimation Platform

Product Design Document

Team Number	Team 8
Platform	Web Application (React + FastAPI)
Backend	Python 3.11, FastAPI, PostgreSQL 15, SQLAlchemy
Frontend	React 18, TypeScript, Three.js, Material-UI 5
Slicer	Ultimaker CuraEngine 5 (Docker)
Version	Release 2
Date	April 2026

Contents

1	Project Overview	2
2	Design Model	2
2.1	Architecture Overview	2
2.2	Class Diagram	3
2.3	Class Descriptions	4
3	Sequence Diagrams	10
3.1	UC1 — User Registration & OTP Email Verification	10
3.2	UC2 — 3D Model Upload & Three.js Viewer	12
3.3	UC3 — Slicing & G-code Generation	13
3.4	UC4 — Cost Estimation	16
4	Design Rationale	19
4.1	Background Task for Slicing (FastAPI BackgroundTasks)	19
4.2	Two-Step Authentication (Registration + OTP Verification)	19
4.3	Build-Volume Pre-Check Before CuraEngine Invocation	20
4.4	Key-Value AdminConfig Table	20
4.5	Frontend State Management — React Context Only	20
4.6	Dual G-code Parsing (Server-side + Client-side)	21

1 Project Overview

The **3D Printing Cost Estimation Platform** is a full-stack web application that allows users to upload 3D model files (STL/STEP), configure slicing parameters, and receive an automated, itemised cost estimate before committing to a physical print. The system integrates Ultimaker CuraEngine 5.0 (running inside a Docker container via subprocess) for G-code generation, a FastAPI backend for business logic and data persistence, and a React 18 + Three.js frontend for 3D model visualisation and user interaction.

Key Capabilities

- **User Registration & OTP Verification** — new accounts require email confirmation via a 6-digit one-time password delivered through the Mailtrap API.
- **3D Model Upload** — supports STL and STEP formats up to 50 MB with server-side format validation, filename sanitisation, and build-volume pre-screening.
- **Interactive 3D Viewer** — Three.js viewer with solid, wireframe, and cross-section (layer clipping) modes, orbital controls, and real-time transform manipulation.
- **Configurable Slicing** — exposes the full CuraEngine 5 parameter surface (layer height, infill density/pattern, wall thickness, support type/density, adhesion, print speed) through a form with quick preset buttons.
- **Automated Cost Estimation** — post-slicing cost engine computes material cost, support material cost, and machine time cost, applying configurable waste and failure overhead multipliers. All costs are denominated in INR.
- **G-code Visualisation** — layer-by-layer Three.js playback of the generated tool path, with animated layer scrubbing.
- **Email Notifications** — asynchronous email alerts for upload success, slicing completion, and slicing failure (with remediation suggestions).
- **Admin Panel** — dedicated dashboard for managing the materials catalogue, editing system configuration parameters, viewing all user jobs, and inspecting failure logs.

2 Design Model

2.1 Architecture Overview

The application follows a classic three-tier architecture with an additional integration layer for the CuraEngine slicer:

React 18 Frontend (TypeScript) $\longleftrightarrow$ FastAPI Backend (Python 3.11)
 $\longleftrightarrow$ PostgreSQL 15

CuraEngine 5.0 (Docker subprocess)



Mailtrap Email API

The backend is organised into four logical layers: *API Routers* (HTTP interface and authorisation), *Services* (core business logic), *ORM Models* (SQLAlchemy entities mapped to PostgreSQL), and *Utilities* (file I/O, validation, parsing).

2.2 Class Diagram

The class diagram below shows all backend ORM models, services, utilities, and API routers alongside the key frontend classes, together with their relationships.

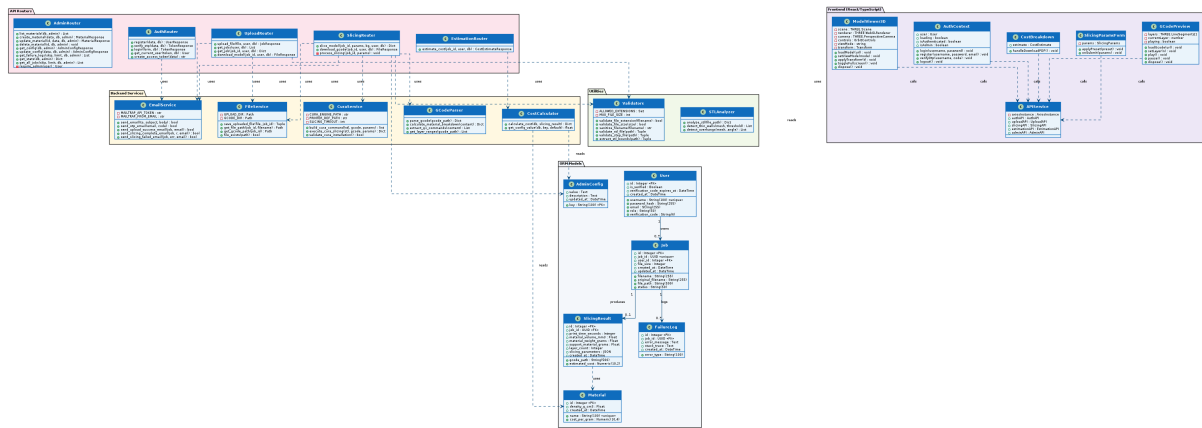


Figure 1: Full class diagram — ORM models, services, utilities, API routers, and frontend components

2.3 Class Descriptions

Class 1 — User (ORM Model / Table: users)

State maintained:

- `id` — auto-increment integer primary key.
- `username` — unique indexed string (max 100 chars); the login identifier.
- `password_hash` — bcrypt-hashed credential (256 chars); plaintext never stored.
- `email` — optional; used for OTP delivery, upload, and slicing notifications.
- `role` — string discriminator: "customer" (default) or "admin".
- `is_verified` — boolean; accounts must pass OTP before any authenticated action is permitted.
- `verification_code` / `verification_code_expires_at` — transient OTP state, cleared after successful verification.
- `created_at` — server-generated timestamp.

Behaviour: This is a pure data-holder entity. Authentication logic (password hashing, JWT creation, OTP validation) lives in the `AuthRouter`. The `require_admin` dependency inspects `role`; `get_current_user` enforces `is_verified` before granting any protected access.

Class 2 — Job (ORM Model / Table: jobs)

State maintained:

- `job_id` — UUID v4; the public-facing identifier exposed to the frontend.
- `user_id` — FK to `users.id`; ownership enforced in all job-scoped endpoints.
- `filename` — normalised storage name (e.g. `model.stl`).
- `original_filename` — user-supplied name; preserved for display and download headers.
- `file_path` — absolute container path to the uploaded model file.
- `file_size` — byte count, validated against the 50 MB ceiling at upload time.
- `status` — lifecycle state: `uploaded` → `slicing` → `sliced` | `failed`.
- `created_at` / `updated_at` — audit timestamps.

Behaviour: Central aggregate linking uploaded files, slicing results, and failure logs. Created by the upload router, mutated by the background slicing task, queried by the estimation and admin routers. Authorisation checks compare `job.user_id` to the requesting user's id, with admins bypassing this restriction.

Class 3 — SlicingResult (ORM Model / Table: slicing_results)

State maintained:

- `job_id` — UUID FK to `jobs.job_id` (one-to-one in practice).
- `gcode_path` — container path to the generated G-code file.
- `print_time_seconds` — extracted from the `;TIME:` G-code comment.
- `material_volume_mm3` — total extruded volume computed by `GCodeParser`.

- `material_weight_grams` — derived: $\text{volume} \div 1000 \times \text{density (g/cm}^3\text{)}$.
- `support_material_grams` — weight of support structures, isolated from `;TYPE:SUPPORT` segments.
- `layer_count` — count of distinct `;LAYER:N` markers in the G-code.
- `estimated_cost` — `Numeric(10,2)` INR total, written by the estimation endpoint.
- `slicing_parameters` — full `SlicingParams` JSON snapshot for audit and cost re-calculation.

Behaviour: Created by `process_slicing` background task after a successful CuraEngine run and G-code parse. The `estimated_cost` field is populated in a second request when the user explicitly triggers `POST /api/estimate/{job_id}`. The stored `slicing_parameters` allow the cost calculator to resolve the correct material without the caller re-supplying it.

Class 4 — Material (ORM Model / Table: materials)

State maintained:

- `name` — unique material identifier (e.g. PLA, ABS, PETG, TPU).
- `density_g_cm3` — used to convert volumetric extrusion data to weight in grams.
- `cost_per_gram` — `Numeric(10,4)` INR unit price applied by the cost calculator.

Behaviour: Managed exclusively through `/api/admin/materials` CRUD (admin-only). Read by `CostCalculator` during estimation and by the slicing pipeline to compute material weight from G-code volume. Seed data pre-populates PLA, ABS, PETG, and TPU.

Class 5 — AdminConfig (ORM Model / Table: admin_config)

State maintained:

- `key` — string PK (e.g. `machine_hourly_rate`, `waste_factor`, `failure_factor`, `printer_volume_x/y/z`).
- `value` — string-serialised value; cast to `float` at read time.
- `description` — human-readable label shown in the admin UI.
- `updated_at` — tracks last configuration change.

Behaviour: Flexible key-value store. Consumed by the slicing pipeline (build-volume limits) and `CostCalculator` (hourly rate, overhead multipliers). Updated via `PUT /api/admin/config`. If a key is absent, consumers fall back to hard-coded defaults, keeping the system tolerant of a fresh or partial configuration table.

Class 6 — FailureLog (ORM Model / Table: failure_logs)

State maintained:

- `job_id` — nullable UUID FK; nullable to support upload-time errors before a job is fully persisted.

- `error_type` — categorical string: `upload_error`, `validation_error`, `slicing_timeout`, `slicing_error`.
- `error_message` — human-readable description of the fault.
- `stack_trace` — optional full Python traceback for unexpected errors.

Behaviour: Append-only audit log written by the upload router (on storage/validation failure) and the slicing background task (on any error type). Surfaced to administrators through `GET /api/admin/logs` with pagination.

Class 7 — CuraService (Service / app/services/cura_service.py)

State: Module-level constants: `CURA_ENGINE_PATH`, `PRINTER_DEF_PATH`, `SLICING_TIMEOUT` (120s).

Public methods:

- `build_cura_command(stl, gcode, params)` — constructs the full CuraEngine slice CLI. Translates application-level enums (infill pattern, adhesion, support type) to CuraEngine 5 setting strings. Model file appended last per the verified CuraEngine 5 argument order.
- `execute_cura_slicing(stl, gcode, params)` — runs the command via `subprocess.run` with `capture_output=True`. Raises `ValueError` on empty-model errors, `TimeoutError` if the process stalls beyond 120s, and `RuntimeError` for any other non-zero exit code.
- `validate_cura_installation()` — startup sanity-check confirming the binary is present.

Class 8 — CostCalculator (Service / app/services/cost_calculator.py)

State: Stateless; reads `Material` and `AdminConfig` from the DB on each call.

Public methods:

- `calculate_cost(db, slicing_result)` — full computation pipeline:
 1. Resolves material from `slicing_parameters["material"]`.
 2. Reads `machine_hourly_rate`, `waste_factor`, `failure_factor` from `AdminConfig`.
 3. **Material cost:** $\text{weight_g} \times \text{cost_per_gram} \times \text{waste_factor} \times \text{failure_factor}$
 4. **Support cost:** same formula applied to `support_material_grams` (if > 0).
 5. **Machine cost:** $(\text{print_time_s} \div 3600) \times \text{machine_hourly_rate}$
 6. Returns itemised breakdown rounded to INR 0.01 using `Decimal` with `ROUND_HALF_UP`.
- `get_config_value(db, key, default)` — helper with fallback for missing config keys.

Class 9 — GCodeParser (Service / app/services/gcode_parser.py)

State: Stateless file-parsing utilities.

Public methods:

- `parse_gcode(gcode_path)` — returns layer count, print time, total material volume, per-type breakdown, and G1 command list.
- `calculate_material_breakdown(content)` — iterates G-code tracking ;TYPE: transitions (SUPPORT, WALL-OUTER, WALL-INNER, FILL, SKIRT, OTHER). Accumulates $\Delta E \times \pi r^2$ per type (filament diameter 1.75 mm).
- `extract_g1_commands(content)` — collects every G0/G1 move as a structured dict {layer, from, to, is_extrusion} for Three.js rendering.
- `get_layer_ranges(gcode_path)` — returns (layer_num, start_line, end_line) for efficient per-layer rendering.

Class 10 — EmailService (Service / app/services/email_service.py)

State: Module-level Mailtrap API credentials from environment; gracefully degrades to console-logging when MAILTRAP_API_TOKEN is absent.

Public methods:

- `send_email(to, subject, html_body)` — async HTTP POST via `httpx.AsyncClient`; returns bool.
- `send_otp_email(email, code)` — dispatches the 6-digit code with 15-minute expiry notice.
- `send_upload_success_email(job, email)` — notifies user of successful upload with next-step instructions.
- `send_slicing_complete_email(job, result, email)` — reports print time, weight, layers, and cost.
- `send_slicing_failed_email(job, error, email)` — delivers actionable remediation advice (thin walls, non-watertight mesh, build-volume violation).

Class 11 — FileService (Service / app/services/file_service.py)

State: Module-level path constants UPLOAD_DIR (/app/uploads) and GCODE_DIR (/app/gcode).

Public methods:

- `save_uploaded_file(file, job_id)` — async streaming write (8 KiB chunks via `aiofiles`) to /app/uploads/{uuid}/model.{ext}; enforces extension allowlist, 50 MB ceiling, and calls `Validators.validate_file_integrity`. Deletes partial file on failure.
- `get_file_path(job_id, filename)` / `get_gcode_path(job_id)` — deterministic path resolution helpers shared across routers.
- `file_exists(path)` — existence guard used before serving file downloads.

Class 12 — STLAnalyzer (Utility / app/utils/stl_analyzer.py)

State: Stateless; uses `trimesh` for mesh operations.

Public methods:

- `analyze_stl(file_path)` — returns volume (mm^3), surface area (mm^2), bounding-box dimensions, centroid, triangle count, watertight status, and a list of detected issues.
- `detect_thin_walls(mesh, threshold=0.8)` — flags any dimension below 0.8 mm.
- `detect_overhangs(mesh, angle_threshold=45.0)` — returns face count and representative locations for faces exceeding the overhang angle.

Class 13 — Validators (Utility / app/utils/validators.py)

State: `ALLOWED_EXTENSIONS = {'.stl', '.step', '.stp'}`; `MAX_FILE_SIZE = 50 MB`.

Public methods:

- `validate_file_extension(filename) / validate_file_size(size)` — guard-clause helpers.
- `sanitize_filename(filename)` — strips path separators and dangerous characters; enforces 255-char limit.
- `validate_stl_file(path)` — checks magic bytes: ASCII STL must contain facet normal; binary STL must have a valid 4-byte triangle count.
- `validate_step_file(path)` — confirms ISO-10303-21 or STEP signature in first 200 bytes.
- `extract_stl_bounds(path)` — pure-Python binary/ASCII STL parser that returns `{min/max/size}` for x, y, z axes without loading trimesh; used for fast build-volume pre-check.

Class 14 — AuthContext (Frontend React Context / context/AuthContext.tsx)

State: `user: User | null`; `loading: boolean`; `isAuthenticated`; `isAdmin` (both derived).

Public methods:

- `login(username, password)` — calls `POST /api/auth/login`; stores token and user in `localStorage`; updates state.
- `register(username, password, email)` — calls `POST /api/auth/register`; login deferred until OTP verification.
- `verifyOtp(username, code)` — calls `POST /api/auth/verify-otp`; on success stores token, completing the login flow.
- `logout()` — clears `localStorage` and resets `user` to null.
- On mount: re-validates any stored token via `GET /api/auth/me`; clears stale tokens on 401.

Class 15 — ModelViewer3D (Frontend Component / `components/ModelViewer3D.tsx`)

State: Three.js scene/renderer/camera/controls/mesh held in React refs; `viewMode` (solid | wireframe | layer); `transform` {posX, posY, posZ, rotX, rotY, rotZ, scale}.

Public methods:

- On `modelUrl` change: loads STL via `THREE.STLLoader`, centres mesh on grid, fits camera to bounding box, starts WebGL animation loop.
- View mode toggles: `MeshPhongMaterial` (solid), `WireframeGeometry` (wireframe), `THREE.Plane` clipping (layer preview).
- Transform panel applies position, rotation (degrees → radians), and uniform scale in real time.
- Fullscreen toggle via the Fullscreen API. Proper teardown on unmount.

Class 16 — GCodePreview (Frontend Component / `components/GCodePreview.tsx`)

State: Per-layer `THREE.LineSegments[]` refs; `currentLayer` / `totalLayers`; `playing` / `speed`.

Behaviour: Fetches the G-code blob from `GET /api/files/{jobId}/output.gcode`; parses G1/G0 commands client-side to construct a `THREE.LineSegments` per layer, coloured by extrusion state (extruding = blue, travel = grey). Layer slider controls visibility; animated playback increments layers at selected speed. Completely independent Three.js scene and lifecycle from `ModelViewer3D`.

Class 17 — APIService (Frontend Service / `services/api.ts`)

State: Axios instance with base URL from `VITE_API_URL`; request interceptor injects Bearer token; response interceptor clears credentials on 401.

Exported namespaces:

- `authAPI` — `register`, `login` (form-encoded), `verifyOtp`, `getMe`.
- `uploadAPI` — `upload` (multipart), `getJobs`, `getJob`, `downloadModel`.
- `slicingAPI` — `slice`, `downloadGcode`.
- `estimationAPI` — `estimate`.
- `adminAPI` — full CRUD for materials, config get/update, logs, stats, all-jobs, system-logs.

3 Sequence Diagrams

Four major use cases are documented below with full sequence diagrams.

3.1 UC1 — User Registration & OTP Email Verification

Actors: User, Frontend (React / AuthContext), Backend (FastAPI), Database (PostgreSQL), Mailtrap Email API

Precondition: User is not registered.

Postcondition: Account created, email verified, JWT token issued, user redirected to Dashboard.

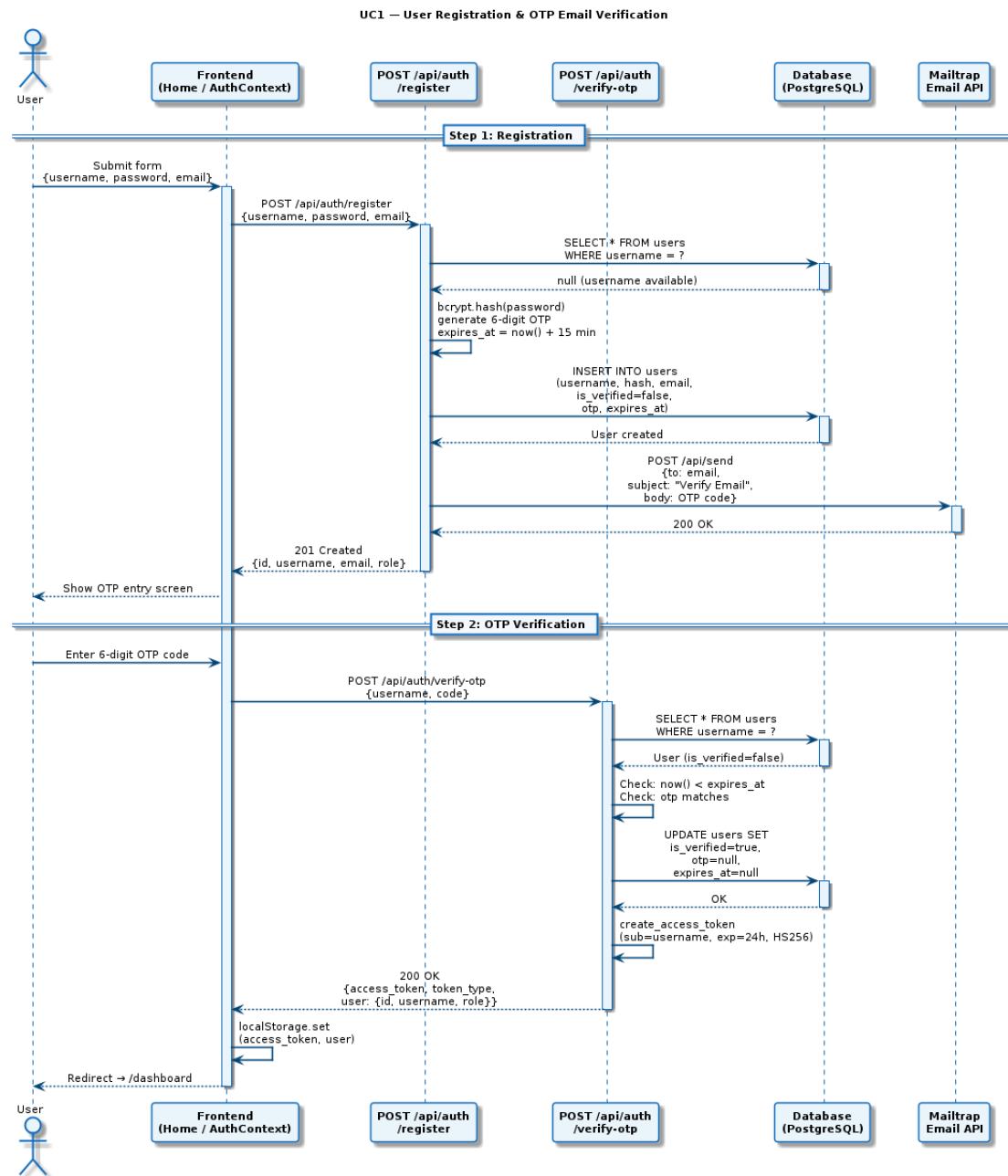


Figure 2: UC1 — User Registration & OTP Email Verification

Narrative: The user submits their credentials on the Home page. The backend hashes the password with bcrypt, generates a cryptographically secure 6-digit OTP with a 15-minute expiry, persists an unverified `User` record, and dispatches the OTP via Mailtrap. The user enters the code; the backend validates it, marks the account as verified, clears the OTP state, and immediately issues a JWT — completing both verification and login in a single response. The frontend stores the token in `localStorage` and redirects to the dashboard.

3.2 UC2 — 3D Model Upload & Three.js Viewer

Actors: Authenticated User, Frontend (FileUploader / ModelViewer3D), Backend, File System

Precondition: User is logged in with a verified account.

Postcondition: Job record created (**status=uploaded**); model rendered in Three.js viewer.

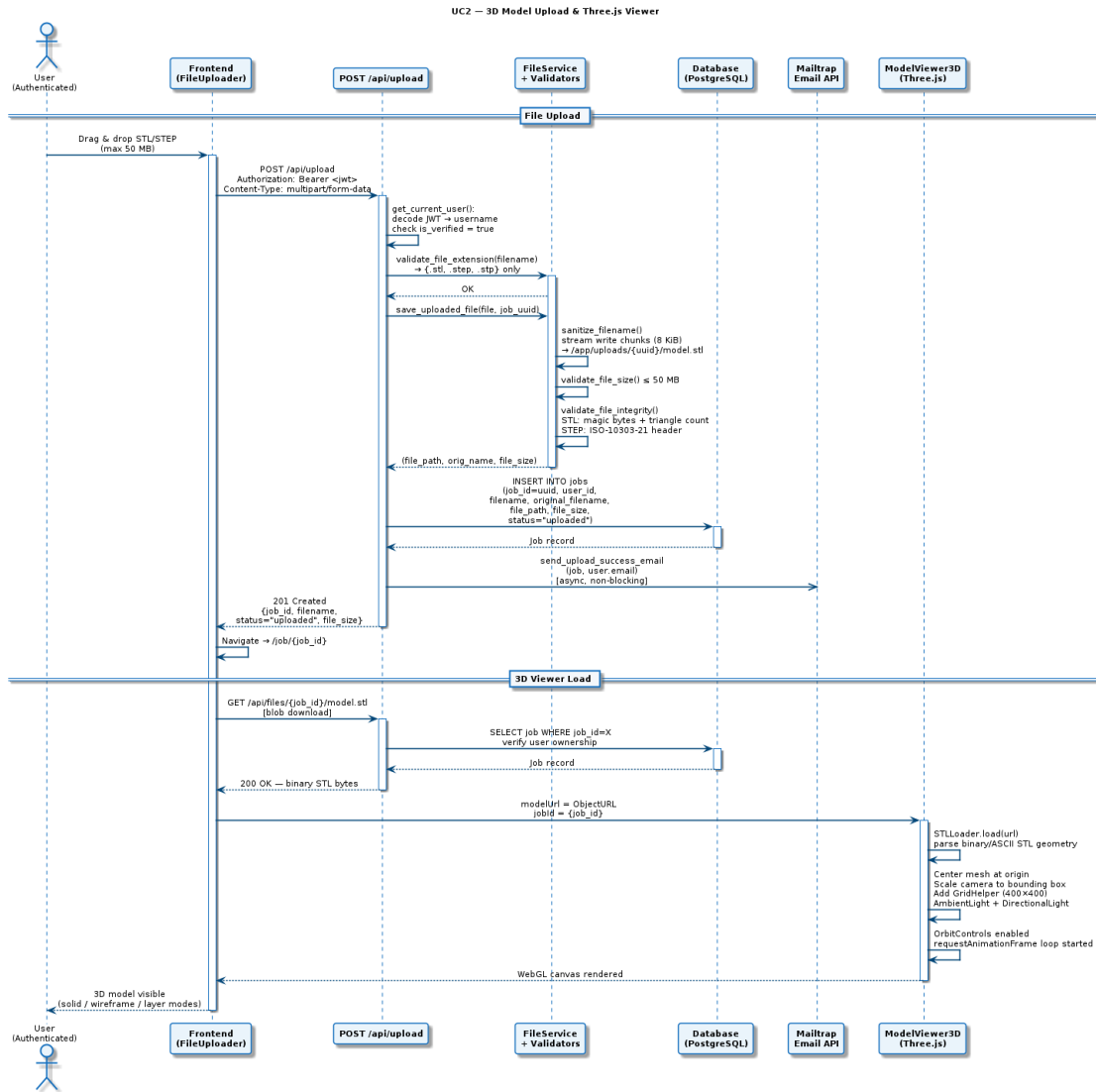


Figure 3: UC2 — 3D Model Upload & Three.js Viewer

Narrative: The user drops an STL/STEP file on the FileUploader dropzone. The backend's `get_current_user` dependency decodes the JWT and enforces `is_verified`. `FileService` streams the upload in 8 KiB chunks (non-blocking via `aiofiles`), validates the extension, size (max 50 MB), and binary format integrity. A `Job` record is committed to PostgreSQL, and an upload-success email is sent asynchronously (non-blocking). The frontend navigates to the Job Detail page, downloads the model blob, and `ModelViewer3D` loads it via `THREE.STLLoader`, centres it on a grid, and starts the orbital render loop.

3.3 UC3 — Slicing & G-code Generation

Actors: User, Frontend (SlicingParamsForm), Backend, CuraEngine 5 (Docker subprocess), GCodeParser, Database, Email

Precondition: Job exists with `status=uploaded`.

Postcondition: `SlicingResult` persisted, `status=sliced`, user notified.

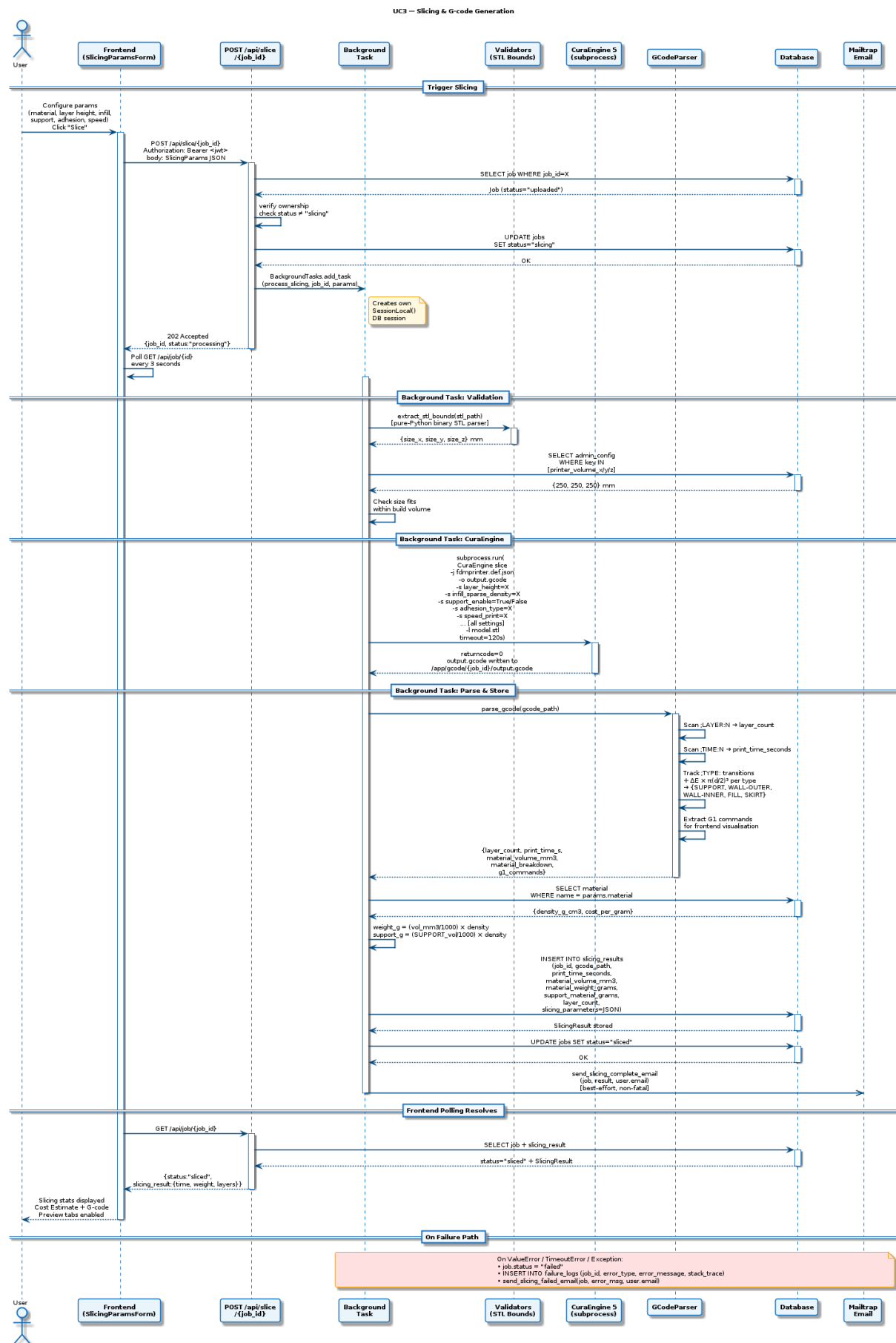


Figure 4: UC3 — Slicing & G-code Generation (including background task and failure path)

Narrative: The user configures slicing parameters and clicks **Slice**. The HTTP handler verifies ownership, sets `status=slicing`, enqueues the background task, and returns `202 Accepted` immediately. The background task runs fully independently with its own database session. It first calls `extract_stl_bounds()` (a pure-Python STL parser) to pre-validate the model against the AdminConfig build volume — rejecting over-sized models instantly without invoking CuraEngine. `CuraService` then assembles the full CLI and runs CuraEngine via `subprocess.run` (120s timeout). On success, `GCodeParser` extracts statistics and material breakdown by tracking `;TYPE:` transitions. A `SlicingResult` is persisted and a completion email sent. On any failure, a `FailureLog` is written, the job marked `failed`, and a failure email with remediation hints sent. The frontend polls every 3 seconds until the status resolves.

3.4 UC4 — Cost Estimation

Actors: User, Frontend (CostBreakdown), Backend (EstimationRouter), CostCalculator, Database

Precondition: Job has `status=sliced` and a `SlicingResult` exists.

Postcondition: `estimated_cost` written to `SlicingResult`; itemised INR breakdown displayed.

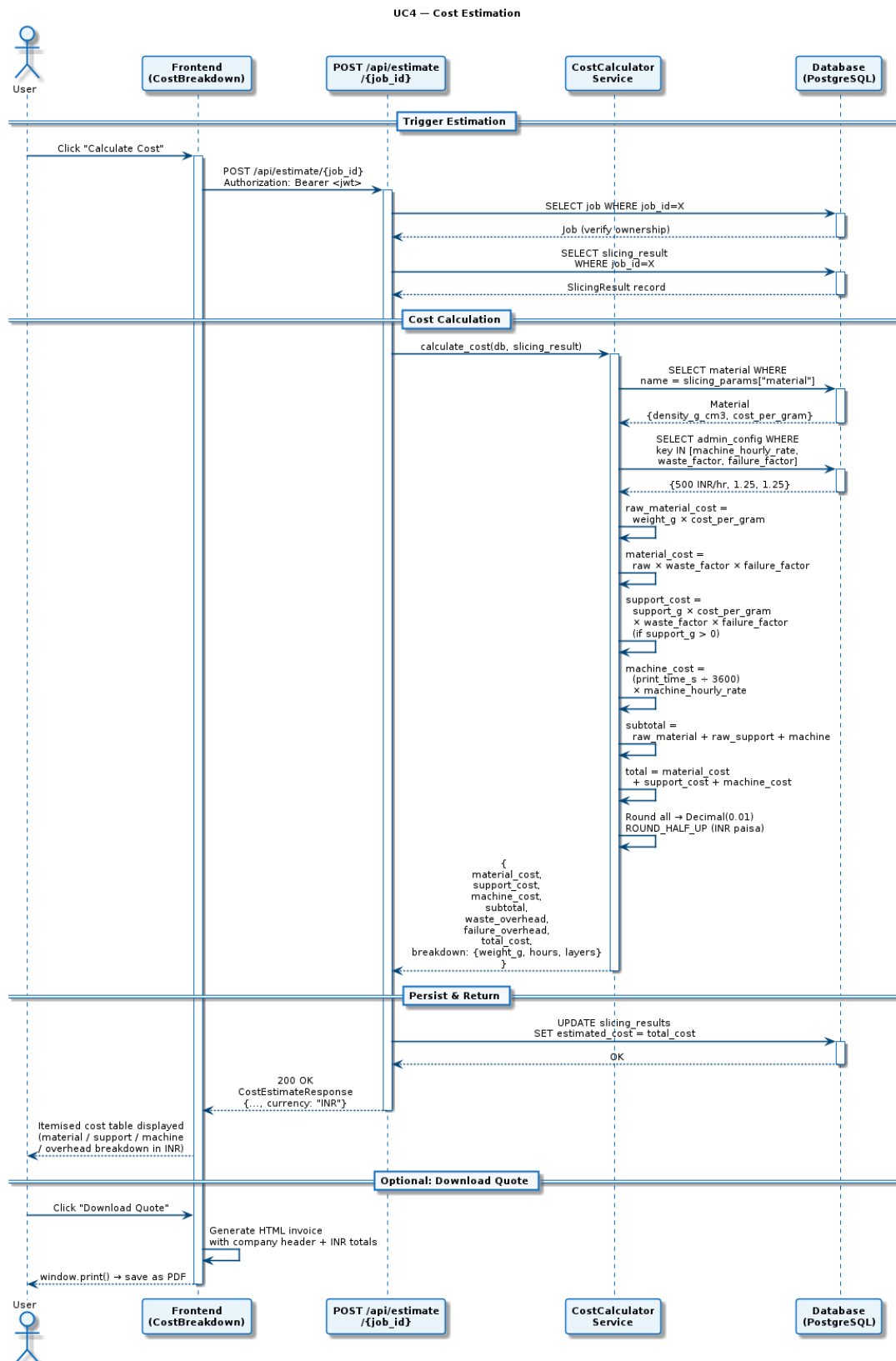


Figure 5: UC4 — Cost Estimation & Quote Download

Narrative: The user clicks **Calculate Cost**. The backend verifies job ownership and retrieves the `SlicingResult`. `CostCalculator` reads the material's `cost_per_gram` and `density_g_cm3` from the `Material` table, and retrieves the current `machine_hourly_rate`, `waste_factor`, and `failure_factor` from `AdminConfig`. It computes material cost, support cost (separately, if applicable), and machine time cost, applying the overhead multipliers. All values are rounded to INR 0.01 using `Decimal` with `ROUND_HALF_UP`. The total is persisted back to `SlicingResult.estimated_cost`. The `CostBreakdown` component renders the full itemised table in INR. The user may also download a print-ready HTML invoice.

4 Design Rationale

4.1 Background Task for Slicing (FastAPI BackgroundTasks)

Problem: CuraEngine slicing takes 15–120 seconds per model. A synchronous HTTP handler would time out in most browser and reverse-proxy configurations and block a worker thread throughout.

Alternatives considered:

1. *Celery + Redis* — production-grade, resumable across restarts, externally monitorable. Adds two new infrastructure components (Celery worker process + Redis broker), significantly increasing deployment complexity for this project's scope.
2. *Synchronous execution with a long timeout* — simple, but unreliable across browser/proxy configurations and blocks a worker thread for the full slicing duration.
3. *FastAPI BackgroundTasks (chosen)* — zero additional infrastructure. The HTTP response is returned immediately (202 Accepted); slicing runs in the same process after the response is sent. The frontend polls for status updates. Trade-off: tasks are not resumable across restarts and cannot be monitored externally, which is acceptable at the current scale.

Key implementation detail: The background task creates its own `SessionLocal()` to avoid sharing the request-scoped session after the HTTP response has been sent.

4.2 Two-Step Authentication (Registration + OTP Verification)

Problem: Prevent throwaway or mistyped email addresses from being registered; ensure email is a valid notification channel for slicing results and cost estimates.

Alternatives considered:

1. *Single-step registration* — simplest, but slicing completion emails would silently fail for invalid addresses.
2. *Magic-link email verification* — better UX, but requires stateless token generation and a public-facing URL that works inside a clicked email, complicating local development.
3. *6-digit OTP with 15-minute expiry (chosen)* — implemented with `secrets.randbelow`; expiry enforced server-side with timezone-aware datetime comparison; OTP state cleared after use. Successful verification auto-issues a JWT, removing a separate login step.

4.3 Build-Volume Pre-Check Before CuraEngine Invocation

Problem: CuraEngine silently generates empty or nonsensical G-code for out-of-bounds models, wasting 30–120 seconds and giving the user a confusing failure with no actionable error.

Alternatives considered:

1. *Parse CuraEngine stderr* — fragile; error message format is not stable across CuraEngine versions.
2. *trimesh bounds extraction* — accurate, but loads the entire mesh into memory and is significantly slower for large binary STLs.
3. *Pure-Python binary STL bounds parser (chosen)* — `extract_stl_bounds()` reads only vertex coordinates from 50-byte triangle records without constructing a full mesh. Runs in under 1 second on a 50 MB file. Build volume limits read from `AdminConfig` so they can be updated by an admin without code changes.

4.4 Key-Value AdminConfig Table

Problem: System parameters (machine rate, overhead factors, printer volume) need to be tunable at runtime by administrators without redeployment.

Alternatives considered:

1. *Environment variables* — changes require container restart; not modifiable via the admin UI.
2. *Strongly-typed config table* — type-safe, but requires a schema migration for every new parameter.
3. *Key-value `admin_config` table (chosen)* — infinitely extensible without migrations. Values stored as `Text`, cast to `float` at read time with a hard-coded fallback. Trade-off: loss of database-level type enforcement, acceptable because all consumers perform explicit `float()` casts.

4.5 Frontend State Management — React Context Only

Problem: Authentication state must be globally shared; admin and customer routes must be cleanly separated; unnecessary global state should be avoided.

Alternatives considered:

1. *Redux Toolkit* — appropriate for complex global state, but introduces boilerplate for an application where only auth state needs to be globally shared.
2. *Zustand* — lightweight alternative, but an additional dependency.
3. *React Context + `localStorage` (chosen)* — `AuthContext` provides `user`, `isAuthenticated`, `isAdmin`, and all auth actions. On mount, any stored token is re-validated against `GET /api/auth/me` to handle expiry. All other data (jobs, slicing results, cost estimates) is fetched locally within each page component.

4.6 Dual G-code Parsing (Server-side + Client-side)

Problem: G-code must be parsed both server-side (for persistent material volume and layer count storage) and client-side (for layer-by-layer Three.js visualisation).

Alternatives considered:

1. *Server-side only, return parsed data via API* — avoids duplicate logic but requires storing and serving potentially very large parsed structures (hundreds of thousands of G1 command dicts).
2. *Client-side only* — reduces server load but means the database cannot store accurate material weight derived from actual extrusion tracking.
3. *Dual parsing (chosen)* — Python `GCodeParser` performs authoritative server-side parsing for persistent storage; the `GCodePreview` React component independently parses the raw G-code file for interactive visualisation. The G-code file itself is the single source of truth, served directly from `GET /api/files/{job_id}/output.gcode`.